

Computer Games Architecture

INM379

Shantanu Bhatt

Part 1:

Loading of assets for game type

Implementation of this is done for loading of score list. It is handled by means of an Content file reader created in FileReader.cs. FileReader.ReadContent takes in a resource location and a type and returns the object of that type at that location.

```
public static T ReadContent<T>(string Location)
{
    try
    {
        T obj = Content.Load<T>(Location);
        return obj;
    }
    catch (Exception e)
    {
        // If we've caught an exception, output an error message
        // describing the error
        Console.WriteLine("ERROR: File could not be found!");
        Console.WriteLine("Exception Message: " + e.Message);
        return default;
    }
}
```

Input System

An input listener and Command Manager is created taking reference from Lab 3 but is modified to be more accessible. The Command Manager is converted into a static class, while the lists to store input keys is modified to store multiple callbacks mapped to the same key. Also another list called ComboBindings is added to add actions that required presses of multiple keys at once. Mouse Input Bindings is also created to handle mouse input.

```
static private InputListener m_Input;
static Dictionary<Keys,List<GameAction>> m_KeyBindings = new Dictionary<Keys, List<GameAction>>();
static Dictionary<List<Keys>,List<GameAction>> m_ComboBindings = new Dictionary<List<Keys>, List<GameAction>>();
static Dictionary<MouseButton,List<MouseAction>> m_MouseBindings=new Dictionary<MouseButton,List<MouseAction>>();
```

Collision Detection

Two types of colliders are added to the game; circle colliders and box colliders. The box collider is Object Oriented Bounding Box (OOBB) implementation. The collision is handed by means of a CollisionManager that stores all the colliders and checks for each collision via brute force techniques.

A Base collider class is created, which is inherited by Circle Collider and Box Collider. These colliders are created in the engine part of the project and can be added to game Objects.

```
public abstract class BaseCollider
{
    public Vector2 position;
    public string tag = "Default";
    public bool isColliding=false;
    public Action<List<HitPoint>> onCollisionEnter;
    public Action onCollisionExit;
    public Action<List<HitPoint>> onCollisionStay;
    5 references
    public abstract void Init(params object[] b);
    6 references
    public abstract void SetCollider(params object[] b);

    3 references
    public abstract void CheckCollision();
}
```

The three actions, OnCollisionEnter, OnCollisionStay, OnCollisionExit are called whenever the respective Collision events occur, they are created referencing Unity's Collision System. It returns a list of hit points, accounting for multiple object collisions.

A hitpoint class is created to return when a collision occurs, that returns important data about the collision to the collided object.

```
public class HitPoint
{
    public Vector2 position;
    public float depth;
    public Vector2 normal;
    public Vector2 penetrationVec;
    public string tag;
}
```

The hit point class contains the hit position. The depth that the hit has passed through each other, the normal of the collision, the Vector that the objects must be moved away from each other to resolve collision and the tag of the other object.

Reference for Circle-Circle and Box-Circle Collision:

<https://web.archive.org/web/20190801002050/https://wildebunny.co.uk/blog/2011/04/20/collision-detection-for-dummies/>

There are three types of collisions to be resolved: Circle-Circle, Box-Circle, Box-Box.

Circle-Circle

Circle-Circle Collision is a radius check, with normal vector being the vector between their centres.

```
public static bool Circle_CircleCollision(CircleCollider c1, CircleCollider c2, out HitPoint hitPoint)
{
    hitPoint = new HitPoint();
    float penetration = (c1.position - c2.position).Length() - (c1.radius + c2.radius);
    if (penetration > 0)
        return false;
    hitPoint.normal = c1.position - c2.position;
    hitPoint.normal.Normalize();
    hitPoint.penetrationVec = -penetration * hitPoint.normal;
    hitPoint.depth = -penetration;
    hitPoint.position = c1.position + (penetration * hitPoint.normal);
    return true;
}
```

Circle-Box Collision

Circle-Box Collision is done by getting the closest box on the box from the sphere. If the distance is less than the radius of the circle the objects are colliding. Normal is the vector between the centre of the circle and the closest point on the box, while the depth vector is the normal multiplied with the distance.

Box-Box Collision

Multiple approaches were used to implement Box-Box Collision. The first approach was the Mikowski Difference implementation. The Minkowski Difference consists of the convex hull of every point on the boundary of one shape subtracted from every point on the boundary of the other shape. It's easier to imagine this as shrinking one of the shapes down until it becomes a point and expanding the other shape by the shape of the first. Configuration space is the space that the Minkowski difference resides within - the set of vector differences of two objects. If the configuration space contains the origin, collision is occurring. [Reference](#)

The second approach was a Diagonal Side intersection implementation, where the intersection between each diagonal of one object is checked with the sides of the other. This approach doesn't account if the objects are small enough that one of the objects is completely inside one of the other so this approach was scrapped.

The approach that is finally implemented is the Separating Axis theorem (SAT). In this approach the sides of both the objects are projected on the normal of their edges and if any of the sides do not overlap, then the objects aren't colliding. The depth vector is determined by adding all the overlaps on each axis. [Reference](#)

Frame Independent Movement

The various objects in the world are created from a class created in the engine known as GameObject. Each Game Object contains a class Transform that contains the position, rotation and scale of each object.

A Class called rigidbody can also be added to each of the gameobjects that moves the object dependent on physics. The rigid body also takes care of collision resolution by taking in the hitpoint values from onCollisionStay and OnCollisionEnter.

Two approaches were applied to simulate physics in the game. The first approach was Euler integration; however, Euler integration causes disparity between the simulated and the realistic physics. To counter this Semi implicit Euler Integration can be applied to reduce the disparity.

However, the Verlet integration method was implemented, that gives very close simulation to realistic physics. In Verlet integration the value of velocity is removed, and velocity is calculated by using the previous and current position of the object. [Reference](#)

```
public void Update(GameTime gameTime)
{
    double dt = gameTime.ElapsedGameTime.TotalSeconds;

    Vector2 vel;
    if (bounceDir.Length() == 0 && !contacted)
        vel = (currentPos - prevPos);
    else
        vel = bounceDir;
    prevPos = currentPos;

    if (contacted)
    {
        currentPos += vel;
        contacted = false;
    }
    else
    {
        currentPos += vel + (impulse+acceleration-(drag*vel)) * (float)(dt * dt);
    }

    bounceDir = Vector2.Zero;
    transform.position = currentPos;

    float rotVel;
    if (rotBounce == 0 && !contacted)
        rotVel = currentRot - prevRot;
    else
        rotVel = rotBounce;
    prevRot = currentRot;

    currentRot += rotVel + (rotAccel-rotDrag*rotVel) * (float)(dt * dt);
    rotBounce = 0;
    transform.rotation = currentRot;

    impulse = Vector2.Zero;
}
```

Part 2

Configurable Game World

The values for the different types of meteors that can be spawned in the world is done by reading from an XML file. A custom list of Meteor Types is created that takes in data required to create the different meteor types. This is then used to create the different meteors required for the world.

```
<?xml version="1.0" encoding="utf-8"?>
<ArrayOfMeteorData xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <MeteorData>
    <scale>
      <X>1</X>
      <Y>1</Y>
    </scale>
    <type>0</type>
  </MeteorData>
  <MeteorData>
    <scale>
      <X>0.75</X>
      <Y>0.75</Y>
    </scale>
    <type>1</type>
  </MeteorData>
  <MeteorData>
    <scale>
      <X>0.5</X>
      <Y>0.5</Y>
    </scale>
    <type>2</type>
  </MeteorData>
</ArrayOfMeteorData>
```

```

public static T ReadXML<T>(string Location)
{
    try
    {
        using StreamReader reader = new StreamReader(Location);
        return (T)new XmlSerializer(typeof(T)).Deserialize(reader.BaseStream);
    }
    catch (Exception e)
    {
        // If we've caught an exception, output an error message
        // describing the error
        Console.WriteLine("ERROR: XML File could not be deserialized!");
        Console.WriteLine("Exception Message: " + e.Message);
        return default(T);
    }
}

```

Removal of game elements based on collision and Event Listeners

Multiple on Collision events are added to the game.

Collision events are handled by subscribing to the onCollisionEnter action that is present in each Collider.

When Player hits a meteor, the meteor is destroyed, and player's health is reduced.

Other instance of collision detection is between bullet and meteor. When a bullet collides with a meteor, an event is fired which checks for the meteors current state and works accordingly.

```

collider.onCollisionEnter += body.OnCollision;
collider.onCollisionEnter += callDestroy;
collider.onCollisionStay += body.OnCollisionStay;
collider.onCollisionExit += body.OnCollisionExit;

```

High Score Table

The high score table is created by saving a list of scores as an XML file. The XML file is first read upon game end and the latest score is added. Then the list is sorted, after which it is clamped to a length of 10 and saved back to the XML file. The display of scores is done by reading the XML File.

```

public static void WriteXML<T>(string Location, T Data)
{
    try
    {
        using (StreamWriter writer = new StreamWriter(Location))
        {
            new XmlSerializer(typeof(T)).Serialize(writer.BaseStream, Data);
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("Error writing XML: " + ex.Message);
        // You can also log the exception or handle it based on your application's requirements
    }
}

```

Part 3

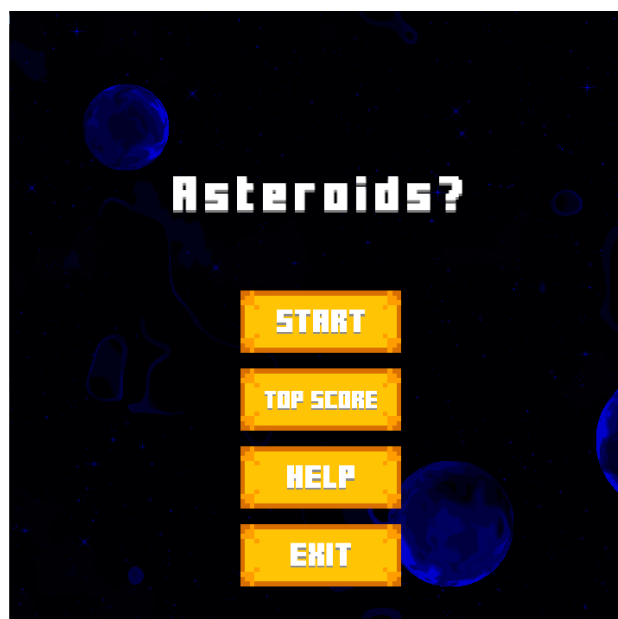
Game Loop FSM

The Game is separated out into multiple states; Help State, Main State, Score State, Game Over State and Start State. The Game State Machine is used to store persistent data between states and any data that is required by multiple states. It also stores the current state and is used to Update and Draw the current state.

The Game State Factory is used to switch between states. The game state factory stores all the states and is used to return a new instance of a state when required.

Reference

The Start State consists of three buttons that are used to switch between the different states and an exit button.



The Button class is an additional class created in the engine referenced from how Unity's Button class works. It can be used to simulate clicks on the screen.

```
internal class Button : GameObject
{
    public Action OnClick;
    Rectangle rect;
    12 references
    public override void Init(params object[] b)
    {
        base.Init(b);
        transform.scale = (Vector2)b[2];
        CommandManager.AddMouseBinding(MouseButton.LEFT, CheckButtonClick);
        CommandManager.AddMouseBinding(CheckMousePos);
        Point scale = new Point((int)(renderer.Width * transform.scale.X), (int)(renderer.Height * transform.scale.Y));
        rect = new Rectangle(transform.position.ToPoint() - new Point(scale.X / 2, scale.Y / 2), scale);
    }

    2 references
    private void CheckButtonClick(eButtonState buttonState, Point position, Point delta)
    {
        if (rect.Contains(position) && buttonState == eButtonState.PRESSED)
            OnClick?.Invoke();
    }

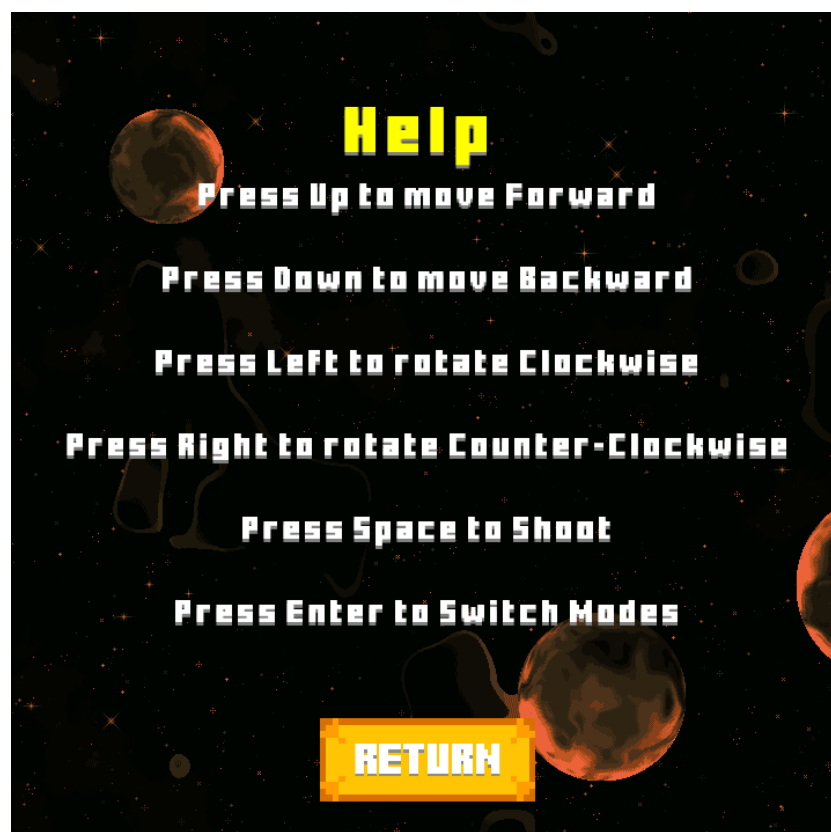
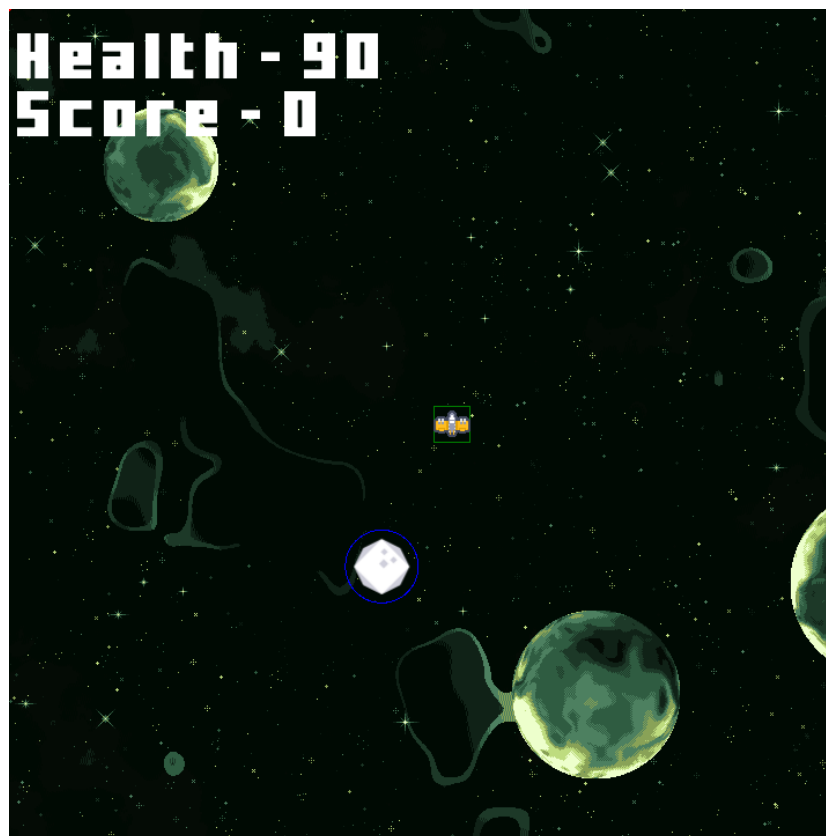
    2 references
    private void CheckMousePos(eButtonState buttonState, Point position, Point delta)
    {
        if (rect.Contains(position))
            renderer.colour = Color.LightGray;
        else
            renderer.colour = Color.White;
    }

    2 references
    public override void Update(GameTime gameTime)
    {
        base.Update(gameTime);
    }

    12 references
    public override void Draw(ref SpriteBatch spriteBatch, GameTime gameTime)
    {
        base.Draw(ref spriteBatch, gameTime);
        //Primitives2D.DrawRectangle(spriteBatch, rect, Color.Red);
    }

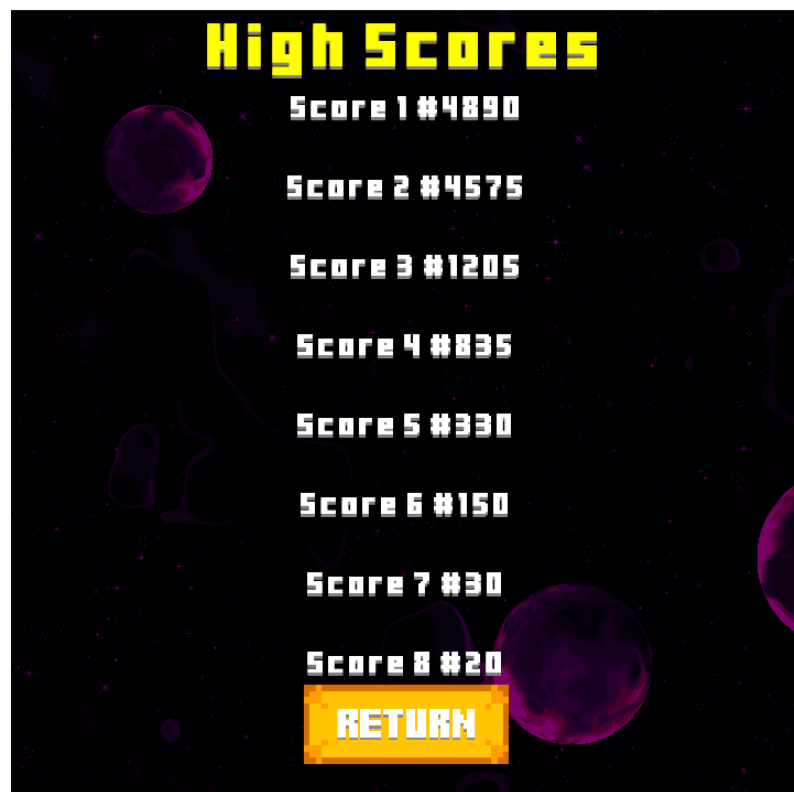
    6 references
    public void DestroyButton()
    {
        CommandManager.RemoveMouseButtons(MouseButton.LEFT, CheckButtonClick);
        CommandManager.RemoveMouseButtons(CheckMousePos);
    }
}
```


Start button switches the current state to the main state, which is the main game loop.



The help button switches the current state to Help State, which displays all the input keys to the game. The return Button is used to go back to the Start State.

The Scores button goes to the Score State displaying the highest 8 scores.



The Main State upon end calls the Game Over State that contains the return button to go back to the start State.

Power Up

A Powerup to change the fire rate of the player has been implemented. Its implementation required changes in the collider to support colliders that do not affect physic collision and are only used as trigger collisions. An event is fired when the Collider is hit with calls the switchshooting function in the player, changing the player's shooting method to full auto. Also Multiple Base Classes such as gameObject, BaseCollider are used to show re -use of different base classes.

Overall Gameplay

The player's task is to survive as long as possible, while trying to score as many points as they can. Each meteor destroyed by the player adds to the score.

The player also has a health meter that gets hit whenever the player collides with a meteor, and dwindling the health to zero causes the game to end. Powerups are spawned at random locations on the map, which the player can collect to change their firing mode to full auto for three seconds.

Part 4

Improvements to the Architecture

The biggest performance hit to the Architecture of the engine is the collision detection. Brute force methods are currently used where each object checking its collision with every other object. This is highly unoptimized as this results in a runtime complexity of $O(n^2)$, which means higher number of objects will slow down the Computation significantly.

One optimization done was to run the Collision logic on multiple threads, increasing performance so that the engine can handle up to 600 collisions smoothly (from the previous 200) in debug mode.

Further improvements include use of QuadTrees, which is a space partitioning structure, however this improvement is only useful if the number of objects being used is high. Also, this implementation uses memory, trading memory for speed, so if memory bottleneck is an issue than it is highly suggested not to use this improvement.

Object pooling can also be used to further speed up the engine, by handling multiple instances of similar objects efficiently.

Assets

1. Button PNG: start PNG Designed By Graphicii from https://pngtree.com/freepng/start-button-in-pixel-art-style_7949383.html?sol=downref&id=bef
2. Background Image: <https://deep-fold.itch.io/space-background-generator>
3. Ship and Meteors: <https://kenney.itch.io/kenney-game-assets> Kenney's Assets
4. Font: <https://www.dafont.com/minercraftory.font>